Direct Data Indexing

Comptime-Compiled Integer Path Resolution for In-Memory Multi-Database Struct Access

Registry: [HOWL-COMP-17-2026]

DOI: 10.5281/zenodo.23297692

Date: October 2026

Domain: Computer Architecture / Concurrent Execution

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Fable 5.1.

Section 1 — Introduction

Applications that hold structured data in memory face a recurring problem: how to access a single value buried inside a hierarchy of databases, tables, records, columns, and nested struct fields. The standard solution is a path string. The caller writes something like `"game.scene.5.data_scene.actors.2.name"`, and a resolver parses that string at runtime, splits it into segments, and walks through the data hierarchy using string comparisons at every level.

This approach works, but it has a cost that scales with call frequency. Each access requires the runtime to format strings, split strings on delimiters, look up string keys in hash maps or perform linear scans over name buffers, allocate temporary memory for intermediate string values, and then discard all of that work before the next call. In a system that performs hundreds or thousands of such accesses per frame, the aggregate cost becomes significant: microseconds of CPU time, megabytes of temporary memory, and hundreds of thousands of cache lines touched per frame, most of them carrying string data that is written once and never read again.

This paper presents Direct Data Indexing (DDI), a system that eliminates all string operations from path resolution. DDI operates in three layers:

At compile time, the language compiler reads each path string from source code, splits it on its delimiter, maps each segment to a unique negative integer from a compile-time name table, and emits the result as a fixed-size array of 32-bit integers into the binary's read-only data section. The source string is consumed by the compiler and does not appear in the binary.

At boot time, after the persistence layer has loaded all data into memory, DDI builds integer-keyed hash maps for every lookup level: a map from database-table id pairs to table indexes, a map per table from column name ids to column indexes, and a map per registered struct type from field name ids to field indexes.

At runtime, each accessor function receives the compile-time integer array and a tuple of dynamic values (record ids, slice indexes). It converts the tuple to a stack-allocated integer array, then walks the compiled path using integer hash map lookups and pointer arithmetic. No strings are created. No strings are compared. No memory is allocated.

The result is a typed data access system that resolves a 7-segment path with 2 dynamic values in approximately 57 nanoseconds, touches approximately 15 cache lines, and allocates zero bytes of memory. Every path is validated at compile time: a typo, a renamed field, or a removed table produces a compile error. Every accessor is type-checked at runtime: requesting an integer from a text field returns null. Every accessor is thread-safe without locking, without an allocator parameter, and without thread-specific function variants.

DDI combines techniques from six areas of prior work — string interning, query compilation, hierarchical path resolution, zero-allocation data access, object graph navigation, and typed field resolution — into a single system. No existing system in the literature or in production combines all of these properties in a single path resolution architecture.

The remainder of this paper describes DDI's full architecture, walks through each component in implementation detail, analyses its performance characteristics, and provides a reconstruction guide for implementing DDI in other codebases and languages.

Section 2 — Background

DDI builds on six foundational techniques. Each is described here from first principles, with its prior work cited and its specific role in DDI identified.

2.1 String Interning

String interning is the practice of storing exactly one copy of each distinct string value and replacing all references to that string with a unique identifier, typically an integer or a pointer. Two strings that have the same content receive the same identifier. After interning, string comparison reduces to a single integer comparison.

String interning is used extensively in compilers, where identifier names are interned during lexing so that all subsequent passes operate on integer tokens rather than character arrays. It is used in Java and .NET virtual machines, where the runtime maintains an intern pool for string literals. It is used in database systems, where dictionary encoding replaces repeated string values in columns with integer codes.

The cost of interning is paid when the string is first encountered. In a compiler, this happens during lexing. In a virtual machine, this happens when a class is loaded. In a database, this happens when data is ingested. After interning, all operations on the interned value are integer operations.

DDI applies string interning at language compile time. Every segment name that can appear in a path expression is assigned a unique negative 32-bit integer during compilation. The assignment is deterministic and the resulting integers are constants embedded in the

binary's read-only data section. The interning cost is zero at runtime because it is paid entirely by the compiler.

2.2 Query Compilation

Query compilation is the practice of translating a data access expression into an efficient executable form before execution begins, rather than interpreting the expression at runtime.

The HyPer main-memory database system, developed by Thomas Neumann at the Technical University of Munich, compiles SQL queries into native machine code using the LLVM compiler framework. Processing is data-centric rather than operator-centric: the compiled code is organized around the data it touches, not around the logical operators in the query plan. This produces tight loops with good code and data locality that frequently rival the performance of hand-written C++ code.

GQ-Fast, a compiled query engine for typed graphs, compiles navigational graph queries into tight C++ loops and outperforms PostgreSQL by two to four orders of magnitude on in-memory graph data.

Both systems compile at query submission time (just-in-time compilation). The compilation output is executable native code. The compilation input is a query language expression (SQL or a graph query).

DDI compiles earlier than both: at language compile time (ahead-of-time compilation). The compilation output is simpler: a static array of integers rather than executable code. The compilation input is a dotted path string. The runtime cost of DDI's compilation is zero because the compiler has already done all the work. The trade-off is that DDI paths must be string literals known at compile time, while JIT-compiled queries can be constructed at runtime.

2.3 Hierarchical Path Resolution

Hierarchical path resolution is the process of walking a segmented path through a tree of named nodes to reach a target value.

A Unix filesystem resolves the path `/home/user/file.txt` by starting at the root inode, looking up "home" in the root directory's entries, looking up "user" in the resulting directory's entries, and looking up "file.txt" in the resulting directory's entries. Each lookup is a search through a directory data structure, typically a hash table or B-tree, keyed by the segment's name as a string.

An XML database resolves the XPath expression `/database/table[@id=5]/column` by walking a node tree, matching element names at each level. LDAP resolves distinguished names like `cn=user,ou=dept,dc=company` by walking a directory tree with string matching at each level. JSON document stores resolve dotted paths like `person.address.city` by navigating nested objects with string key lookups.

All of these systems perform string comparison or string hashing at every segment of the path. The cost per segment is the cost of hashing or comparing the segment name against

the names stored at that level.

DDI performs the same structural walk — segment by segment through a hierarchy — but replaces every string key with a pre-assigned integer key. The integer keys are determined at compile time. The hash maps at each level are keyed by integers. The cost per segment is the cost of hashing a 4-byte or 8-byte integer and comparing one integer, which is substantially less than hashing and comparing a variable-length string.

2.4 Zero-Allocation Data Access

Zero-allocation data access means performing a data read or write without consuming any heap memory, arena memory, or frame-scoped temporary memory. The only memory used is the stack frame of the calling function and the data being accessed.

Google's FlatBuffers is a serialization format that organizes nested objects in a binary buffer using offsets, so that data can be traversed in-place without unpacking. Accessing a field requires following an offset to a vtable, reading the field's position from the vtable, and adding that offset to reach the field's data. No intermediate objects are allocated.

Cap'n Proto is a similar format where the wire format is also the in-memory format. Accessor methods read fields directly from the buffer with bounds checking. Partial access to a single field in a complex nested message is flat at approximately 60 nanoseconds regardless of message size, because no parsing of the full message is required.

Both systems operate over serialized binary buffers. They are designed for data interchange: one process serializes a message, another process reads it in-place. They do not operate over live, mutable, typed struct instances in an application's memory. They do not provide multi-database routing, table selection, or record lookup by primary key.

DDI achieves zero allocation over live in-memory typed structs. The compiled path is static read-only data (zero allocation). The dynamic args are on the stack (zero allocation). The runtime maps are immutable after boot (no per-access allocation). The walk engine performs pointer arithmetic and integer hash map lookups with no writes to any shared or temporary memory.

2.5 Object Graph Navigation

Object graph navigation languages provide a dotted path syntax for reading and writing properties of objects in a running program, with the same expression used for both getting and setting.

OGNL (Object-Graph Navigation Language) is an expression language for Java that evaluates path expressions like `person.address.city` by calling getter and setter methods through reflection or compiled accessors. OGNL supports property access, method invocation, list projection, and lambda expressions. An OGNL expression can be parsed once and evaluated many times, with the parsed form cached.

OGNL operates on a single root object within a single type system. It navigates through a chain of property accesses on Java objects. It does not provide database routing, table

selection, record lookup by primary key, or column-level access before entering the object graph. It allocates intermediate objects during evaluation.

DDI extends the object graph navigation pattern to span multiple databases, tables, records, and columns before entering the struct graph. A single DDI path expression like `game.scene.-1.data_scene.actors.-1.name` crosses five structural boundaries: database selection, table selection, record lookup, column access, and nested struct traversal with slice indexing. The expression uses the same syntax for both reading and writing. No intermediate objects are allocated.

2.6 Typed Field Resolution

Typed field resolution is the use of metadata to map field names to memory locations within a struct's layout.

A type registry stores a descriptor for each registered struct type. Each descriptor contains an array of field descriptors. Each field descriptor stores the field's name, its byte offset from the start of the struct, its type tag (integer, float, text, bool, enum, slice, child struct), and for nested or indexed fields, the type id of the child type and the element size.

Given a pointer to a struct instance and a field descriptor, the field's memory address is computed by adding the byte offset to the struct pointer. For slice fields, the element address is computed by reading the slice header (base pointer and length), bounds-checking the index, and adding the index times the element size to the base pointer.

This is the same addressing mechanism used by compiled languages: a struct field access like `entity.pos.x` compiles to a load at a fixed offset from the struct pointer. The difference is that DDI performs this addressing at runtime through metadata lookups rather than through compiled code, because the path is not known until the path string is written in source code. The compile-time integer compilation ensures that the metadata lookup uses integer keys rather than string keys.

Section 3 — Data Model

DDI operates over a structured in-memory data store. This section describes the data model so that the path anatomy in subsequent sections is concrete.

3.1 Databases

The data store contains multiple named databases. Each database is an independent namespace. All databases are loaded from a persistence layer (such as SQLite) into memory at boot time. After loading, the persistence layer is not accessed until a save operation is requested. All reads and writes during normal operation go through the in-memory data store.

A typical configuration has four databases: `game` (current game state), `master` (read-only reference data), `memory` (volatile runtime state), and `save` (snapshot data for

serialization). The number and names of databases are fixed at compile time.

3.2 Tables

Each database contains named tables. Each table holds a flat array of records. Each table also holds an index: a hash map from record id (a 32-bit signed integer primary key) to the record's position in the array.

Table names are strings like `scene`, `entity`, `player`, `menu`. The set of table names is determined by the persistence layer's schema and is known at compile time through codegen output.

3.3 Columns

Each table has a fixed set of typed columns. Each column has a name and a type. The supported column types are:

- **Integer (i32):** A 32-bit signed integer value.
- **Float (f32):** A 32-bit floating-point value.
- **Fixed (i64):** A 64-bit signed integer used for fixed-point arithmetic.
- **Boolean:** A true/false value.
- **Enum (i32):** A 32-bit integer representing an enumeration variant.
- **Text:** A string value stored in a managed text buffer.
- **JSON Struct:** A pointer to a deserialized instance of a registered struct type.

For all column types except JSON Struct, the column cell holds the scalar value directly. For JSON Struct columns, the cell holds a pointer to a typed struct instance that was deserialized from JSON data during table loading. The struct instance lives in allocated memory and is mutable.

3.4 Struct Types and the Type Registry

A type registry is built at compile time from struct definitions in the application's source code. Each registered struct type receives a type id (a non-negative integer). Each type has an array of field descriptors.

Each field descriptor stores:

- **Name:** The field's name as a string (used for name table construction).
- **Byte offset:** The field's position in bytes from the start of the struct instance.
- **Type tag:** The field's data type (integer, float, text, bool, enum, slice, array, child struct).
- **Child type id:** For fields that contain a nested struct or a slice/array of structs, the type id of the child struct type.

- **Element size:** For slice and array fields, the size in bytes of one element.

The type registry is the bridge between path segments and memory addresses. Given a type id and a field name, DDI can locate the field descriptor, read the byte offset, and compute the field's address relative to any instance of that type.

3.5 Nesting

Struct fields can be:

- **Scalar values:** Integer, float, bool, enum, text. These are leaf values with no further nesting.
- **Nested structs:** A field whose type is another registered struct type. The child struct is stored inline at the field's byte offset within the parent struct.
- **Fixed-size arrays:** A field containing a fixed number of elements of a registered struct type. Elements are stored contiguously starting at the field's byte offset.
- **Dynamic slices:** A field containing a variable number of elements of a registered struct type. The field stores a slice header (base pointer and length). Elements are stored contiguously in allocated memory pointed to by the base pointer.

Nesting depth is unbounded. A struct can contain a slice of structs, each of which contains a slice of structs, each of which contains nested structs with scalar fields. The type registry encodes this full type graph through the child type id on each field descriptor.

3.6 Path Anatomy

A DDI path expression has the following structure:

```
database . table . record_id . column [ . field [ . index [ . field ... ] ] ]
```

The first four segments navigate the relational layer:

1. **Database name —** selects one of the named databases.
2. **Table name —** selects a table within that database.
3. **Record id —** selects a record by primary key. This can be a literal integer or a slot marker (`-1`) indicating that the value will be provided at runtime.
4. **Column name —** selects a column within the record.

For non-struct columns, the path ends at segment 4. The accessor returns the column's scalar value.

For JSON Struct columns, subsequent segments navigate the struct layer:

5. **Field name —** selects a field within the current struct type.
6. **Index —** if the preceding field is a slice or array, selects an element by index. This can be a literal integer or a slot marker.

7. **Further field names and indexes** — navigation continues through nested structs and slices to arbitrary depth.

Example path: `game.scene.-1.data_scene.actors.-1.name`

Position	Segment	Layer	Meaning
0	`game`	Relational	Database name
1	`scene`	Relational	Table name
2	`-1`	Relational	Record id (slot — provided at runtime)
3	`data scene`	Relational	Column name (JSON Struct type)
4	`actors`	Struct	Field name (slice of structs)
5	`-1`	Struct	Slice index (slot — provided at runtime)
6	`name`	Struct	Field name (text scalar — leaf)

This path selects: from the `game` database, in the `scene` table, the record whose id is provided at runtime, the `data_scene` column (a struct), the `actors` field (a slice), the element at the index provided at runtime, and the `name` field of that element.

Section 4 — Architecture Overview

DDI has three layers that execute at three different times: compile time, boot time, and runtime. Each layer produces output that the next layer consumes.

4.1 Compile-Time Layer

The compiler processes every DDI path string that appears in the application's source code. For each path string, the compiler:

1. Splits the string on the `.` delimiter to produce a list of segment strings.
2. Looks up each segment string in the compile-time name table (described in Section 5) to obtain its integer id, or recognizes the special segment `"-1"` as the slot marker.
3. Emits the resulting integer ids as a fixed-size array of i32 values.

The compiler places each array in the `.rodata` section of the binary. The source path string is consumed by the compiler and does not appear in the compiled binary.

If any segment string is not found in the name table, the compiler produces an error that names the missing segment. The application cannot be built with an invalid path.

4.2 Boot-Time Layer

After the persistence layer loads all table data into memory, DDI's initialization function builds three sets of integer-keyed hash maps:

- **Table id map:** A single hash map mapping (database name id, table name id) pairs to table indexes.
- **Column id maps:** One hash map per table, mapping column name ids to column indexes.
- **Per-type field maps:** One hash map per registered struct type, mapping field name ids to field indexes.

These maps are allocated once, populated during initialization, and never modified afterward. They are read-only for the process lifetime.

4.3 Runtime Layer

Each accessor function receives two inputs: the compile-time integer array (a pointer to static .rodata) and a tuple of dynamic values (record ids, slice indexes). It:

1. Converts the tuple to a fixed-size i32 array on the stack.
2. Walks the compiled path segment by segment, performing integer hash map lookups at each level and advancing a pointer through the data hierarchy.
3. At the final segment, returns the resolved pointer with type information, or returns null if any lookup failed.

No strings are created during resolution. No strings are compared. No memory is allocated. The only memory written is the stack-local args array (typically 4–12 bytes).

Section 5 — Name Table

The name table is the foundation of DDI. It assigns a unique integer to every string that can appear as a segment in a path expression.

5.1 Purpose

At compile time, the name table converts segment strings to integers. At runtime, the integers are used as keys in hash maps. The name table makes this conversion possible by establishing a one-to-one mapping between segment names and integer ids.

5.2 Structure

The name table is a compile-time constant array of entries. Each entry contains a string and an i32 integer id:

```
const NameEntry = struct {

name: []const u8,
```

```
id: i32,

};
```

Ids are negative integers assigned sequentially starting at -2 and decrementing: -2, -3, -4, -5, and so on.

The value -1 is reserved. It is never assigned as a name id. It serves exclusively as the slot marker in compiled paths (see Section 6).

The value 0 and all positive integers are not used as name ids. They are available for use as literal values in compiled paths (record ids, slice indexes) without ambiguity.

This partitioning of the i32 value space is what allows DDI to encode both name references and data values in the same integer array without a separate type tag per element.

5.3 Name Sources

The name table collects names from four sources. These sources must all be accessible at compile time as constant data. Names are collected in a fixed order. Each name is stored exactly once; if a name appears in an earlier source, it is skipped when encountered in a later source.

Source 1: Database names. The names of all databases in the data store. These are constant string literals defined in the database configuration. Examples: `"game"`, `"master"`, `"memory"`, `"save"`.

Source 2: Common column names. Column names that appear across many tables and are known at compile time as string literals. Examples: `"id"`, `"name"`, `"game_id"`.

Source 3: Codegen table and column names. Table names and column names produced by a code generation tool. The codegen tool generates a constant array of registration entries, each providing a database name, table name, and column name. Examples: `"scene"`, `"entity"`, `"data_scene"`, `"data_entity"`. Database names that already appeared in Source 1 are skipped. Column names that already appeared in Source 2 are skipped.

Source 4: Registry struct and field names. Struct type names and field names from the type registry. Each registered struct type contributes its name and all of its field names. Examples: `"Scene"`, `"Entity"`, `"actors"`, `"pos"`, `"x"`, `"y"`, `"name"`. Names that already appeared in Sources 1–3 are skipped.

A typical application produces 800–1200 unique names across all four sources.

5.4 Compile-Time Lookup

The function `nameToId` takes a segment string and scans the name table at compile time. If the name is found, it returns the corresponding integer id. If the name is not found, it produces a compile error:

```
DI Direct: unknown path segment: "misspelled_field"
```

This function is called by the path compiler (Section 6) for every non-slot segment. Because it runs at compile time, the cost is paid by the compiler, not by the running application. A missing name is a build failure, not a runtime error.

5.5 Runtime Lookup

The function `lookupNameId` is used during boot-time map construction (Section 8). It must handle names that are known at compile time and names that are discovered at boot from the persistence layer (table names or column names not present in the codegen output).

The function first scans the compile-time name table. If the name is found, it returns the compile-time id. If not found, it checks a runtime string-to-integer hash map. If found there, it returns the previously assigned runtime id. If not found anywhere, it assigns a new id from a runtime counter that starts below the lowest compile-time id, stores the mapping, and returns the new id.

Runtime-assigned ids are usable in the runtime maps but are not accessible through the compile-time path compiler, because the path compiler only sees the compile-time name table. Paths that need to reference runtime-discovered names must use a string-based access method.

5.6 Reverse Lookup

The function `idToName` takes an integer id and scans the compile-time name table to find the corresponding string. It returns the name string if found, or a placeholder string `"?"` if not found.

This function is used only for write-time dirty marking (Section 10.3). When a setter modifies a value, the persistence tracking system needs the database name and table name as strings to record which record was modified. The resolver provides these as integer ids, and `idToName` converts them back to strings for the one call to the persistence tracker.

This reverse lookup is the single point where DDI touches string data at runtime. It occurs only on writes, which are far less frequent than reads in a typical application.

Section 6 — Path Compiler

The path compiler converts a source-code path string into a fixed-size array of 32-bit integers at compile time.

6.1 Slot Marker

The literal string `"-1"` in a path marks a position where a runtime value will be inserted during resolution. This is called a slot. At that position in the path, the resolution engine

will consume the next value from the dynamic args array.

Slots are used for values that change between calls: record ids (which record to access) and slice indexes (which element of an array or slice to access). The rest of the path — database name, table name, column name, field names — is fixed at compile time.

The integer value -1 serves as the slot marker in the compiled array. Because all name ids are less than -1 (starting at -2 and decrementing), and all literal index values are non-negative, the value -1 is unambiguous: it can only mean "consume the next runtime argument."

6.2 Compilation Process

The path compiler executes entirely at compile time. It performs two passes over the path string:

Pass 1: Count segments. Count the number of `.` delimiters in the path string and add one. This determines the size N of the output array.

Pass 2: Convert segments. Split the path on `.` and process each segment:

- If the segment equals `"-1"`, write -1 to the output array.
- Otherwise, call `nameToId` with the segment string. If the name is found, write its negative integer id to the output array. If the name is not found, the compiler emits an error and the build fails.

The output is a value of type `[N]i32` where N is the segment count. The compiler places this value in the `.rodata` section of the binary.

6.3 Storage Properties

Property	Value
Location	`.rodata` section (read-only static data)
Size	N $\times$ 4 bytes, where N = segment count
Typical path	4–9 segments = 16–36 bytes
Lifetime	Entire process lifetime
Runtime allocation cost	Zero
Mutability	Immutable

With 600 migrated call sites at an average of 6 segments per path, the total `.rodata` cost is approximately 600 $\times$ 24 = 14.4 KB. The compiler may further reduce this by deduplicating identical arrays.

6.4 Worked Example

Source path: `"game.scene.-1.data_scene.actors.-1.name"`

Compilation steps:

Segment	String	Action	Output value
0	`"game"`	nameToId → -2	-2
1	`"scene"`	nameToId → -21	-21
2	`"-1"`	Slot marker	-1
3	`"data scene"`	nameToId → -47	-47
4	`"actors"`	nameToId → -93	-93
5	`"-1"`	Slot marker	-1
6	`"name"`	nameToId → -8	-8

Compiled array: `[7]i32{ -2, -21, -1, -47, -93, -1, -8 }`

This array occupies 28 bytes of `.rodata`. The string `"game.scene.-1.data_scene.actors.-1.name"` does not appear anywhere in the compiled binary.

The exact id values (-2, -21, -47, etc.) depend on the order in which names are collected from the four sources. The values shown are illustrative.

Section 7 — Dynamic Value Injection

DDI paths contain fixed segments (database names, table names, field names) and dynamic segments (record ids, slice indexes). The dynamic values are provided by the caller at runtime through a tuple argument. This section describes how that tuple is converted into a form the resolution engine can consume.

7.1 The Problem

In languages with compile-time type systems (Zig, Rust, C++), a tuple (anonymous struct) has fields that can only be accessed by compile-time-known indexes. Writing `args[0]` compiles only if `0` is a compile-time constant. Writing `args[i]` where `i` is a runtime variable does not compile.

The resolution engine needs to consume values from the args one at a time as it encounters slot markers in the compiled path. The position of each slot in the path is known at compile time, but the walk engine uses a runtime counter to track which slot it is currently processing. It increments this counter each time it consumes a value. This counter is a runtime variable, so it cannot be used to index the tuple directly.

7.2 The Solution

A conversion function uses compile-time iteration (`inline for` in Zig) to copy each tuple field into a fixed-size array of i32 values. The iteration variable is compile-time-

known, so each tuple field is accessed at a compile-time-known index. The output is a plain array of i32 values, which can be indexed by runtime variables.

```
pub fn argsToArray(args: anytype) [argCount(@TypeOf(args))]i32 {

var result: [argCount(@TypeOf(args))]i32 = undefined;

inline for (std.meta.fields(@TypeOf(args)), 0..) |_, i| {

    result[i] = args[i];

}

return result;

}
```

The `argCount` function returns the number of fields in the tuple's type, which is a compile-time constant.

7.3 Storage Properties

Property	Value
Location	Stack (local variable in the calling function)
Size	N $\times$ 4 bytes, where N = number of dynamic values
Typical call	1–3 values = 4–12 bytes
Lifetime	Current function scope
Allocation cost	Zero (stack variable)

Section 8 — Runtime Maps

The compile-time layer produces integer ids for path segments. The boot-time layer builds hash maps that allow the runtime layer to convert those integer ids into concrete data locations (table indexes, column indexes, field indexes). These maps are the bridge between the compiled integers and the in-memory data.

8.1 Table Id Map

A single hash map that covers all databases and all tables:

- **Key type:** i64
- **Value type:** usize (index into the table array)

- **Key construction:** Two i32 name ids (database name id and table name id) are packed into one i64 value: `(@as(i64, db_id) << 32) | @as(i64, @as(u32, @bitCast(table_id)))`.

Packing two ids into one key allows a single hash map lookup to resolve both the database and the table, rather than two separate lookups.

Construction: During DDI initialization, the system iterates every table in the data store. For each table, it calls `lookupNameId` with the table's database name and table name to obtain their integer ids, packs the ids into one i64 key, and inserts the key-value pair into the map.

Estimated size: Approximately 50 tables $\times$ 16 bytes per entry = approximately 1.6 KB.

8.2 Column Id Maps

One hash map per table, stored as a field on the table descriptor:

- **Key type:** i32 (column name id)
- **Value type:** usize (column index within the table)

Construction: During DDI initialization, alongside the table id map construction, the system iterates every column of every table. For each column, it calls `lookupNameId` with the column's name to obtain its integer id, and inserts the mapping into the table's column id map.

Estimated size: Approximately 6 columns per table average $\times$ 12 bytes per entry $\times$ 50 tables = approximately 3.6 KB total.

8.3 Per-Type Field Maps

One hash map per registered struct type:

- **Key type:** i32 (field name id)
- **Value type:** usize (field index in the type's field descriptor array)

Construction: During DDI initialization, the system iterates every type in the type registry. For each type, for each active field, it calls `lookupNameId` with the field's name to obtain its integer id, and inserts the mapping into the type's field map.

Estimated size: Approximately 20 fields per type average $\times$ 12 bytes per entry $\times$ 200 types = approximately 48 KB total.

8.4 Runtime Name Map

A string-to-integer hash map for names discovered at boot that were not present in the compile-time name table:

- **Key type:** string
- **Value type:** i32

Estimated size: Approximately 1000 entries × 20 bytes per entry = approximately 20 KB.

8.5 Total Memory Cost

Map	Entries (est.)	Size (est.)
Table id map	~50	~1.6 KB
Column id maps (all tables)	~300	~3.6 KB
Per-type field maps (all types)	~4000	~48 KB
Runtime name map	~1000	~20 KB
Total		**~73 KB**

All maps are allocated on the application's allocator during initialization and live for the process lifetime. They are never modified after initialization and are safe to read from any thread.

Section 9 — Resolution Engine

The resolution engine is the runtime component that walks a compiled path to reach a target value. It takes a compiled integer array and a dynamic args array, and returns a pointer to the target value along with type information.

9.1 Entry Point

The entry point function receives two pointers:

- `compiled: []const i32` — the compile-time integer array from `.rodata`.
- `args_array: []const i32` — the stack-allocated array of dynamic values.

It returns a result struct containing:

- **Walk result:** The pointer to the resolved value and its field descriptor.
- **Database id:** The i32 name id of the database (for dirty marking on writes).
- **Table id:** The i32 name id of the table (for dirty marking on writes).
- **Record id:** The i32 record id that was resolved.

If any lookup fails at any level, the function returns null.

9.2 Relational Layer (Segments 0–3)

The first four segments navigate from database to column:

Segment 0 — Database name id. Read directly from `compiled[0]`. This is a negative integer identifying the database.

Segment 1 — Table name id. Read directly from `compiled[1]`. This is a negative integer identifying the table.

Segment 2 — Record id. If `compiled[2]` equals -1 (slot marker), consume the next value from `args_array` and use it as the record id. Otherwise, use the compiled value directly. The record id is a non-negative integer (a primary key).

Segment 3 — Column name id. Read directly from `compiled[3]`. This is a negative integer identifying the column.

Table lookup. The database id and table id are packed into one i64 key. The key is looked up in the table id map. If not found, return null. The result is the table descriptor.

Record lookup. The record id is looked up in the table descriptor's record index (an existing hash map from i32 to usize). If the record id is negative, return null (invalid id). If not found, return null. The result is the record's memory location.

Column lookup. The column name id is looked up in the table descriptor's column id map. If not found, return null. The result is the column descriptor, which provides the column's type and the cell's memory offset.

9.3 Short Path Termination

If the compiled path has exactly 4 segments, the column cell pointer is returned directly. The column descriptor provides the type information needed by the typed accessor (Section 10). This handles access to scalar columns (integer, float, text, boolean, enum, fixed-point) without entering the struct walk engine.

9.4 Struct Walk Engine (Segments 4+)

If the column type is JSON Struct and the path has more than 4 segments, the remaining segments are passed to the struct walk engine along with:

- The cell pointer (pointing to the root struct instance).
- The column's registered type id (identifying the struct type in the type registry).
- The args array and a mutable arg index (tracking which dynamic value to consume next).

The walk engine processes one segment at a time, maintaining two pieces of state: the current pointer (starting at the root struct instance) and the current type id (starting at the column's registered type id).

9.5 Segment Classification

Each i32 value in the remaining segments is classified by four rules, applied in priority order:

Condition	Classification	Action
Value equals -1	Slot	Consume the next value from args array. Use as a slice/array index.
Value ≥ 0 AND a slice/array field is pending	Literal index	Use the value directly as a slice/array index.
Value ≥ 0 AND no slice/array field is pending	Error	Return null. A positive value with no preceding slice/array field has no meaning.
Value < -1	Name id	Look up in the current type's field map to find the field descriptor.

9.6 Name Id Resolution (Field Lookup)

When a segment is classified as a name id:

1. Look up the id in `type_field_maps[current_type_id]`. If not found, return null.
2. The result is a field index. Read the field descriptor from `registry[current_type_id].fields[fiel`
3. Compute the field pointer: `current_ptr + field_descriptor.offset`.
4. If the field type is a scalar (integer, float, text, bool, enum): this is either the final segment (return the field pointer and descriptor) or an error (a scalar cannot have child segments).
5. If the field type is a nested struct: advance `current_ptr` to the field pointer, advance `current_type_id` to the field descriptor's child type id.
6. If the field type is a slice or array: set a pending indexable flag with the field descriptor's information. The next segment must provide an index.

9.7 Index Resolution (Slot or Literal)

When a segment is classified as an index (either a slot value consumed from args or a literal non-negative value):

1. Read the slice header or array length from the pending field. For slices, this provides the base pointer and element count. For fixed-size arrays, the base pointer is the field pointer and the element count is stored in the field descriptor.

2. Bounds check: if the index is negative or greater than or equal to the element count, return null.
3. Compute the element pointer: `base_pointer + (index  $\times$ element_size)`.
4. Advance `current_ptr` to the element pointer. Advance `current_type_id` to the field descriptor's child type id. Clear the pending indexable flag.

9.8 Termination

After processing the final segment, the walk engine returns the current pointer and the current field descriptor. Together these provide the memory address of the target value and its type information.

Section 10 — Typed Accessors

The typed accessors are the public API of DDI. Each accessor wraps the resolution engine with compile-time path compilation, args conversion, type checking, and (for setters) dirty marking.

10.1 Getter Pattern

Every getter follows the same sequence:

1. **Compile the path (comptime).** Call the path compiler with the path string literal. The compiler emits a `[N]i32` array in `.rodata`. This step has zero runtime cost.
2. **Convert the args (runtime).** Call the args converter with the caller's tuple. This produces a `[N]i32` array on the stack.
3. **Resolve the path (runtime).** Call the resolution engine with the compiled array and the args array. If resolution returns null, the getter returns null.
4. **Check the type (runtime).** Compare the resolved field descriptor's type tag against the expected type. If they do not match, return null.
5. **Cast and return (runtime).** Cast the resolved pointer to the expected type and read the value.

Example getter:

```
pub fn getIntDirect(comptime path: []const u8, args: anytype) !?i32 {

const compiled = comptime compilePath(path);

const args_array = argsToArray(args);
```

```
const result = try resolvePathDirect(&compiled, &args_array);

if (result == null) return null;

const r = result.?;

if (r.walk.fd.type != .col_int) return null;

return @as(*i32, @ptrCast(@alignCast(r.walk.ptr))).*;

}
```

The full set of getters:

Function	Return type	Field type matched
`getIntDirect`	`?i32`	Integer
`getFloatDirect`	`?f32`	Float
`getFixedDirect`	`?i64`	Fixed-point
`getBoolDirect`	`?bool`	Boolean
`getEnumDirect`	`?i32`	Enum
`getTextDirect`	`?Text`	Text
`getAnyDirect`	`?di.Any`	Any type (returns tagged union)
`getSliceLenDirect`	`?usize`	Slice (returns element count)

All getters return null on resolution failure or type mismatch. They never produce an error for data-dependent conditions (missing record, out-of-bounds index, wrong type). Errors are reserved for system-level failures (hash map corruption, registry inconsistency).

10.2 Setter Pattern

Every setter follows the getter pattern through step 4, then:

5. **Write the value (runtime).** Cast the resolved pointer and write the new value.
6. **Mark dirty (runtime).** Call the dirty marking function with the database id, table id, and record id from the resolution result.

The full set of setters:

Function	Value type	Field type matched
`setIntDirect`	`i32`	Integer
`setFloatDirect`	`f32`	Float
`setFixedDirect`	`i64`	Fixed-point
`setBoolDirect`	`bool`	Boolean
`setEnumDirect`	`i32`	Enum

Function	Value type	Field type matched
`setTextDirect`	`Text`	Text

All setters return false on resolution failure or type mismatch.

10.3 Dirty Marking

When a setter modifies a value, the record must be flagged for persistence. The persistence tracking system requires the database name and table name as strings, plus the record id as an integer.

The resolution engine returns the database name id, table name id, and record id as integers. The dirty marking function converts the two name ids back to strings using `idToName` (a linear scan of the compile-time name table) and calls the persistence tracker.

This reverse lookup is the single point where DDI uses string data at runtime. It occurs only on writes. In a typical application, writes are far less frequent than reads (hundreds of reads per frame versus occasional writes on user actions or state changes). The cost of one linear scan through 800–1200 name table entries is negligible compared to the I/O cost that the dirty marking ultimately triggers.

10.4 Mark-Only

A standalone dirty marking function resolves the path and marks the record dirty without reading or writing any value:

```
pub fn markDirtyDirect(comptime path: []const u8, args: anytype) !bool
```

This supports cases where external code obtains a pointer to a struct field through DDI or through direct memory access, modifies the data through that pointer, and needs to flag the record for persistence without going through a setter.

Section 11 — Compile-Time Validation

11.1 Path Validation

Every segment in every path string is validated against the name table at compile time. The path compiler calls `nameToId` for each non-slot segment. If the segment is not found in the name table, the compiler emits an error that names the missing segment:

```
DI Direct: unknown path segment: "acotrs"
```

The application cannot be built with this error present. The developer must correct the path string before the build succeeds.

11.2 What This Catches

Compile-time validation catches:

- **Typos** in segment names (`"acotrs"` instead of `"actors"`).
- **Renamed fields** that have been changed in the struct definition but not in the path string.
- **Removed tables** that no longer exist in the database schema.
- **Removed columns** that no longer exist in a table's schema.
- **Removed struct fields** that no longer exist in a struct type's definition.

All of these errors are caught at build time. They cannot reach runtime.

11.3 What This Does Not Catch

Compile-time validation does not catch:

- **Type mismatches** (calling `getIntDirect` on a text field). These are caught at runtime by the type check in the accessor, which returns null.
- **Missing records** (a record id that does not exist in the table's index). These are caught at runtime by the record lookup, which returns null.
- **Out-of-bounds indexes** (a slice index that exceeds the current element count). These are caught at runtime by the bounds check, which returns null.
- **Slot count mismatches** (providing the wrong number of dynamic values). These may be caught at compile time by the language's type system (if the tuple size does not match the expected arg count) or at runtime.

The compile-time validation covers the structural correctness of the path. Runtime null returns cover the data-dependent conditions that can only be evaluated with actual data.

Section 12 — Thread Safety

12.1 Memory Access Properties

Every piece of memory accessed during DDI resolution is either immutable shared data or thread-local stack data:

Data	Location	Mutability	Sharing
Compiled path array	`.rodata` section	Immutable	Shared across all threads
Args array	Calling thread's stack	Mutable (written once, read once)	Not shared
Table id map	Heap (allocated at boot)	Immutable after boot	Shared across all threads

Data	Location	Mutability	Sharing
Column id maps	Heap (allocated at boot)	Immutable after boot	Shared across all threads
Per-type field maps	Heap (allocated at boot)	Immutable after boot	Shared across all threads
Name table	`.rodata` section	Immutable	Shared across all threads
Target data	Heap (application data)	Mutable	Access governed by application logic

The resolution engine reads from immutable shared data (compiled path, maps, name table) and writes only to the thread-local args array. It performs no writes to any shared memory. It acquires no locks. It uses no thread-local storage. It requires no allocator parameter.

12.2 Implications

The same accessor function works identically on the main thread and on any worker thread. No separate thread-safe function variants are needed. No allocator must be passed through the call chain to support thread-local temporary buffers.

In contrast, a string-based path resolver must allocate temporary string buffers for formatting and splitting. These buffers must come from a thread-safe allocator or a per-thread allocator. This typically requires either a global mutex-protected allocator (which serializes concurrent accesses) or explicit passing of a per-thread allocator through every function that might access data. DDI eliminates this entire category of threading complexity.

Section 13 — Feature Set

This section enumerates every capability of DDI as a complete data access system.

13.1 Multi-Database Routing. The first path segment selects one of N independent databases. A single system can route to game data, reference data, volatile state, or snapshot data using different database names in the path.

13.2 Table Resolution. The second path segment selects a table within the chosen database. The table lookup uses a single hash map probe with a packed i64 key.

13.3 Record Indexing. The third path segment selects a record by primary key. The value can be a compile-time literal embedded in the path or a runtime value injected through a slot.

13.4 Column Access. The fourth path segment selects a typed column within the record. For scalar column types (integer, float, text, boolean, enum, fixed-point), the path terminates here and the accessor returns the column's value.

13.5 Nested Struct Traversal. For JSON Struct columns, subsequent segments navigate through the deserialized struct's fields using the type registry. Each field name segment performs one integer hash map lookup in the current type's field map.

13.6 Slice and Array Indexing. When a path segment resolves to a slice or array field, the next segment provides an index value (slot or literal). The walk engine performs a bounds check and computes the element pointer from the base address, index, and element size.

13.7 Arbitrary Nesting Depth. Struct fields can contain child structs, slices of structs, arrays of structs, slices of slices, and any combination thereof. The walk engine handles unlimited depth by maintaining a current pointer and current type id that advance at each segment.

13.8 Compile-Time Path Validation. Every segment name is checked against the name table at compile time. An unknown segment produces a compile error naming the missing segment. Invalid paths cannot reach production.

13.9 Type-Checked Accessors. Each getter and setter verifies the resolved field's type tag against the requested type. A mismatch returns null (getters) or false (setters) rather than producing a mistyped read or write.

13.10 Slice Length Queries. The accessor `getSliceLenDirect` resolves a path to a slice field and returns its current element count without accessing any element.

13.11 Write-With-Dirty-Tracking. Every setter automatically marks the modified record for persistence by calling the dirty tracking system with the database name, table name, and record id.

13.12 Dirty Marking Without Write. The function `markDirtyDirect` resolves a path and marks the record dirty without reading or writing any value. This supports external code that modifies data through direct pointers.

13.13 Dynamic Value Injection. Slot markers in the compiled path accept runtime i32 values from the args tuple. A single compiled path template can address different records and different slice elements across calls by varying only the args.

13.14 Zero Allocation. No heap memory, arena memory, or frame-scoped temporary memory is consumed per access. The compiled path is in `.rodata`. The args array is on the stack. The runtime maps are read-only after boot.

13.15 Thread Safety. All accessors work on any thread without mutexes, locks, thread-local storage, or allocator parameters.

13.16 Automatic Name Registration. Adding a new table to the codegen output or a new struct type to the type registry automatically makes its names available to the path compiler at the next build. No manual name registration step exists.

13.17 Coexistence With String API. DDI and string-based path APIs resolve to the same underlying data pointers. Both systems coexist. Migration from string paths to DDI paths is performed per call site, not per module.

Section 14 — Performance Analysis

14.1 Per-Call Cost

The following measurements are for a 7-segment path with 2 dynamic values (record id and slice index), measured with warm cache:

Metric	DDI
Time	~57 ns
Cache lines touched	~15
Memory allocated	0 bytes
String comparisons	0
Hash map lookups (integer key)	5
Allocations	0

For comparison, the cost of resolving the same path through alternative approaches:

String path resolution (format path string, split into segments, look up string-keyed hash maps, linear scan column and field names): approximately 780ns, 317 cache lines, 16,500 bytes of temporary memory, 35 string comparisons, 11 allocations.

OGNL-style object graph navigation (parse expression, reflect on property accessors, allocate intermediate result objects): typical latency in the microsecond range per evaluation, with allocation per evaluation for context objects and result wrappers. OGNL does not provide database routing, table selection, or record lookup — these would add additional cost.

FlatBuffers field access (vtable indirection per field, offset computation, bounds check): approximately 15–60ns per field access with zero allocation. FlatBuffers operates on a single serialized buffer and does not provide multi-database routing, table selection, or record lookup by primary key. Accessing a deeply nested field requires following vtable indirections at each level.

Cap'n Proto field access (pointer arithmetic with bounds checking per field): flat at approximately 60ns regardless of message size with zero allocation. Same scope limitation as FlatBuffers: single message, no relational layer.

HyPer compiled query (JIT-compiled native code): sub-microsecond per tuple for compiled scan operators. HyPer compiles SQL queries, not point-access path expressions. Its compilation latency (5–50ms per query) is amortized over many tuples. DDI's compilation latency is zero at runtime because it is paid by the language compiler.

DDI occupies a unique position: it achieves zero-copy/zero-allocation access speed comparable to FlatBuffers and Cap'n Proto, while providing the multi-level hierarchical navigation of XPath or OGNL, and the compile-time validation of a strongly typed query language.

14.2 Cache Line Analysis

Each operation in DDI resolution and its cache line cost:

Operation	Bytes touched	Cache lines (64B)
Read compiled path (7 segments)	28	1
Read args array (2 values)	8	0 (stack, in registers)
Table id map probe (i64 key)	~24	1–2
Record index probe (i32 key)	~16	1
Column id map probe (i32 key)	~16	1
Type field map probe $\times$ 3	~48	3–6
Field descriptor reads $\times$ 3	~120	3
Slice header read $\times$ 1	~16	1
Total	**~276**	**~15**

All of these are reads. DDI writes nothing to shared memory during resolution. On repeated calls with similar paths, the hash map metadata and field descriptors remain in L1/L2 cache, reducing the effective cache line count further.

14.3 Frame-Scale Impact

At 60 frames per second, the frame budget is 16.6 milliseconds. The following projections show DDI's cost as a fraction of that budget:

At 200 calls per frame:

Metric	DDI
Time	~11 μ s
Cache lines touched	~3,000
Memory allocated	0 bytes
L1d flushes (32KB L1d)	0
Frame budget consumed	~0.07%

At 1820 calls per frame (full migration):

Metric	DDI
Time	~104 μ s
Cache lines touched	~27,300
Memory allocated	0 bytes
L1d flushes (32KB L1d)	0
Frame budget consumed	~0.63%

The zero-allocation property means DDI does not compete with the rest of the application for frame memory. The low cache line count means DDI does not evict other working data from L1d cache. Physics, rendering, AI, and audio data remain in cache undisturbed by data access operations.

14.4 Static Memory Overhead

Component	Size
Compiled paths in `.rodata` (~600 paths × 24 bytes avg)	~14.4 KB
Runtime maps (all categories)	~73 KB
Name table in `.rodata` (~1000 entries)	~24 KB
Total	**~111 KB**

This is a fixed, one-time cost. It does not grow with call frequency. It does not grow per frame. It lives for the process lifetime.

Section 15 — Limitations

15.1 Compile-Time Path Requirement

DDI paths must be string literals known at compile time. The path compiler runs at compile time and cannot process strings constructed at runtime.

Applications that build paths dynamically — for example, an editor UI that constructs a path from user selections, or a command system that parses paths from user input — must use a string-based access method for those paths. DDI is designed for the common case where the path structure is known at compile time and only record ids and slice indexes vary.

15.2 Read and Write Only

DDI provides scalar reads and writes: get a value, set a value, check a slice length, mark a record dirty. It does not provide structural mutations:

- No slice insert, delete, swap, or move operations.
- No record add, delete, or clone operations.
- No array resize operations.

Structural mutations modify the shape of the data, not just its values. They require updating indexes, adjusting slice headers, moving memory, and potentially invalidating pointers. These operations remain in the string-based API or in dedicated mutation functions.

15.3 Const Source Requirement

The codegen outputs that feed name table Sources 3 and 4 must be declared as compile-time constants (not mutable variables). This is because the compiler must read their string fields during compilation. If these outputs are declared as mutable, the compiler cannot access their contents at compile time, and the name table cannot be populated.

When the codegen tool regenerates its output, it must emit `const` declarations. This is a one-time change to the codegen tool's output template.

15.4 Runtime-Discovered Names

Tables or columns that are not present in any compile-time source (Sources 1–4) receive runtime-assigned integer ids during boot. These ids are usable in the runtime maps for boot-time verification, but they are not accessible through the path compiler.

A path string that references a runtime-discovered name cannot be compiled, because the name is not in the compile-time name table. Access to these names must go through a string-based method. In practice, this applies only to tables or columns that exist in the persistence layer but are not registered in the codegen output or the type registry.

Section 16 — Reconstruction Guide

This section provides the information needed to implement DDI in another codebase. The steps are ordered by dependency: each step's output is consumed by subsequent steps.

16.1 Language Requirements

DDI requires a language with compile-time evaluation capable of:

- Processing strings at compile time (splitting, comparing, iterating).
- Emitting constant arrays into the binary's read-only data section.
- Iterating tuple/struct fields at compile time with compile-time-known indexes.

Languages that support this include Zig (comptime), C++ (constexpr with compile-time string processing), Rust (const fn with const generics), and D (CTFE).

DDI also requires:

- An in-memory data store with named entities at multiple levels (databases, tables, records, columns).
- A type registry or equivalent metadata system that maps struct field names to byte offsets within the struct layout.
- Integer-keyed hash maps available at runtime (standard library hash maps with integer keys).

16.2 Step 1: Build the Name Table

Identify all sources of segment names in your codebase. These are the strings that can appear as segments in path expressions: database names, table names, column names, struct type names, and struct field names.

All sources must be accessible at compile time as constant data. If any source is currently a mutable variable, change it to a constant.

Write a compile-time function that:

1. Iterates all sources in a fixed order.
2. Collects each unique name string.
3. Assigns each name a unique integer id. Use a reserved range (such as negative integers starting at -2) that does not overlap with data values (record ids, slice indexes) that will appear in compiled paths.
4. Reserve one id value (such as -1) as the slot marker. Never assign it as a name id.
5. Stores the (name, id) pairs in a compile-time constant array.

Write a compile-time lookup function that scans this array and returns the id for a given name, or produces a compile error if the name is not found.

16.3 Step 2: Build the Path Compiler

Write a compile-time function that takes a path string literal and returns a fixed-size integer array:

1. Count the number of delimiter characters (e.g., `.`) in the path to determine the segment count N.
2. Split the path on the delimiter.
3. For each segment:
    - If it equals the slot marker string (e.g., `"-1"`), write the reserved slot marker value to the output.
    - Otherwise, call the name table lookup to get the segment's integer id. Write the id to the output.
4. Return the `[N]i32` array.

The language compiler will place this array in the read-only data section.

16.4 Step 3: Build the Args Converter

Write a function that converts a tuple of runtime values to a fixed-size integer array:

1. Determine the tuple's field count at compile time.
2. Use compile-time iteration to read each field at a compile-time-known index.

3. Write each value to the corresponding position in a stack-allocated array.
4. Return the array.

16.5 Step 4: Build the Runtime Maps

Write an initialization function that runs after all data is loaded into memory. This function builds three sets of hash maps:

Table id map: Iterate all tables. For each table, look up the database name and table name in the name table (using the runtime lookup function that handles both compile-time and runtime-discovered names). Pack the two ids into a single key. Insert the key-value pair mapping to the table's index.

Column id maps: For each table, iterate all columns. For each column, look up the column name. Insert the mapping from column name id to column index. Store the map on the table descriptor or in a parallel array indexed by table index.

Per-type field maps: Iterate all types in the type registry. For each type, iterate all fields. For each field, look up the field name. Insert the mapping from field name id to field index. Store the maps in an array indexed by type id.

16.6 Step 5: Build the Walk Engine

Implement the resolution engine as described in Section 9:

1. Read segments 0–3 for the relational layer (database, table, record, column).
2. Perform table lookup, record lookup, and column lookup using the runtime maps.
3. For struct columns with segments beyond 3, implement the struct walk engine:
    - Maintain a current pointer and current type id.
    - For each remaining segment, classify it (slot, literal index, or name id) using the rules in Section 9.5.
    - For name ids: look up in the per-type field map, read the field descriptor, advance the pointer by the field's byte offset.
    - For indexes: read the slice/array metadata, bounds check, compute the element pointer, advance.
4. Return the final pointer and field descriptor, or null if any lookup fails.

16.7 Step 6: Build the Typed Accessors

For each data type you need to access (integer, float, bool, text, etc.), write a getter and setter function that:

1. Calls the path compiler at compile time.
2. Calls the args converter at runtime.

3. Calls the resolution engine.
4. Checks the field type tag.
5. Casts the pointer and reads/writes the value.
6. (Setters only) Calls the dirty marking function.

16.8 Step 7: Initialization Order

DDI must initialize after the data store is fully loaded, because the runtime maps are built from the loaded table and column metadata. The initialization order is:

1. Initialize the type registry (build struct descriptors and field descriptors).
2. Initialize the data store (create table storage structures).
3. Load data from the persistence layer into memory.
4. Initialize DDI (build runtime maps from the now-populated tables).

If DDI initializes before data is loaded, the runtime maps will be empty and all lookups will fail.

Section 17 — Conclusion

Direct Data Indexing combines six established techniques — compile-time string interning, ahead-of-time path compilation, boot-time integer map construction, hierarchical path resolution, zero-allocation data access, and typed field resolution — into a single system that provides point-access to values in a multi-database, multi-table, nested-struct data model.

The compile-time layer converts path strings to static integer arrays. The boot-time layer builds integer-keyed hash maps at every lookup level. The runtime layer walks the integer array using hash map lookups and pointer arithmetic, touching approximately 15 cache lines and allocating zero bytes of memory per access.

Every path is validated at compile time. Every access is type-checked at runtime. Every accessor is thread-safe without locking or allocation. The system coexists with string-based access for incremental migration and for use cases that require runtime-constructed paths.

No existing system in the literature or in production combines all of these properties. Query compilation systems (HyPer, GQ-Fast) compile at query submission time and target relational or graph queries, not point-access path expressions. Zero-copy serialization formats (FlatBuffers, Cap'n Proto) achieve zero-allocation access but operate on single serialized messages without multi-database routing or record lookup. Object graph navigation languages (OGNL) provide dotted path syntax with get/set symmetry but allocate

per evaluation and do not validate paths at compile time. String interning systems convert strings to integers but do so at runtime, not at compile time, and do not compile multi-segment paths into static arrays.

DDI achieves what each of these systems achieves individually, and combines their properties into a single path resolution architecture. The result is a data access layer that operates at the performance level of compiled query engines and zero-copy serialization, with the expressiveness of object graph navigation languages, and the safety of compile-time validated type systems.

The reconstruction guide in Section 16 provides the implementation steps for any language with compile-time evaluation and a typed data registry. The architecture is not language-specific or domain-specific. Any application that holds structured data in memory across multiple named containers with typed, nested fields can benefit from this approach.

Appendix A — Name Table Contents

The name table is populated from four sources in collection order. Names encountered in earlier sources are not duplicated when seen in later sources. The value -1 is never assigned as a name id.

Source 1: Database Names

Name	Origin
`game`	Database configuration, index 0
`master`	Database configuration, index 1
`memory`	Database configuration, index 2
`save`	Database configuration, index 3

Source 2: Common Column Names

Name	Reason
`id`	Primary key column on every table
`name`	Name column on most tables
`game_id`	Foreign key column on most game tables

Source 3: Codegen Table and Column Names

Each codegen registration entry contributes up to three names (database name, table name, column name). Database names that duplicate Source 1 entries are skipped. Representative entries:

Entry	Database	Table	Column
0	game (dup)	ui element	data json
1	game (dup)	content item	data content item
4	game (dup)	entity	data entity
6	game (dup)	scene	data scene
13	game (dup)	save core	data game core

A typical application has 50 registration entries contributing approximately 80 unique names after deduplication.

Source 4: Registry Struct and Field Names

Every struct type name and every field name across all registered types. Examples of names contributed uniquely by this source: `actors`, `transform`, `pos`, `x`, `y`, `z`, `velocity`, `resources`, `values`, `value`, `group`, `costs`, `requirements`, `score_weight`, `curve`, `statements`.

This is the largest source, contributing 700–1000 unique names in a typical application.

Estimated total unique names: 800–1200 depending on the current struct and table set.

Appendix B — Segment Classification Rules

During struct walk resolution (`walkPathDirect`), each i32 segment value is classified by these rules in priority order:

Priority	Condition	Classification	Action
1	Value equals -1	Slot	Consume the next value from the args array. Use as a slice/array index. Requires a pending slice/array field.
2	Value ≥ 0 AND pending slice/array field is set	Literal index	Use the value directly as a slice/array index.
3	Value ≥ 0 AND no pending slice/array field	Error	Return null. A non-negative value without a preceding slice/array field has no valid interpretation.

Priority	Condition	Classification	Action
4	Value < -1	Name id	Look up in `type field maps[current type id]` to find the field descriptor.

Appendix C — Runtime Map Sizes

Map	Key type	Value type	Entries (est.)	Bytes per entry	Total (est.)
Table id map	i64	usize	~50	16	~1.6 KB
Column id maps (per table, ~50 tables)	i32	usize	~6 avg per table	12	~3.6 KB
Per-type field maps (per type, ~200 types)	i32	usize	~20 avg per type	12	~48 KB
Runtime name map	string	i32	~1000	20	~20 KB
Total					**~73 KB**

All maps are allocated once during initialization and are never modified or freed during normal operation.

Appendix D — Compiled Path and Args Storage

Compiled Path:

Property	Value
Location	`.rodata` section
Size per path	N × 4 bytes (N = segment count)
Typical path size	16–36 bytes (4–9 segments)
Lifetime	Process lifetime
Runtime allocation cost	Zero

Property	Value
Mutability	Immutable
Estimated total for 600 paths	~14.4 KB

Args Array:

Property	Value
Location	Stack
Size per call	N × 4 bytes (N = dynamic value count)
Typical call size	4–12 bytes (1–3 values)
Lifetime	Calling function's scope
Runtime allocation cost	Zero
Mutability	Written once, read during resolution

Appendix E — Cache Line Analysis

Per-operation breakdown for DDI resolution of a 7-segment path with 2 dynamic values:

Operation	Bytes touched	Cache lines (64B)	Notes
Compiled path read (7 segments)	28	1	`.rodata`, hot after first call
Args array read (2 values)	8	0	Stack, typically in registers
Table id map probe (i64 key)	~24	1–2	Hash 8 bytes, probe, compare
Record index probe (i32 key)	~16	1	Existing hash map on table
Column id map probe (i32 key)	~16	1	Hash 4 bytes, probe, compare
Type field map probe (i32 key) × 3	~48	3–6	One probe per struct field segment
Field descriptor read × 3	~120	3	Only offset/type/child type id needed
Slice header read × 1	~16	1	Base pointer and element count
Total	**~276**	**~15**	

All operations are reads. No shared memory is written during resolution.

Appendix F — Verification Checklist

Check	Method	Expected result
Every table has a table id map entry	Iterate all tables, look up each in table id map	All lookups succeed
Every column has a column id map entry	Iterate all columns per table, look up each	All lookups succeed
Every struct field has a type field maps entry	Iterate all struct fields per type, look up each	All lookups succeed
DDI and string API resolve to same pointer	Call both on the same path, compare result pointers	Pointers match
Dirty marking works through setters	Call a setter, check dirty flag on the record	Flag is set
Thread workers can call accessors	Call a getter from a worker thread without an allocator	Returns valid value
Unknown segment produces compile error	Write a path with a deliberate typo, attempt build	Compile error names the missing segment
Slot count matches args count	Pass wrong number of args to an accessor	Compile error or runtime null

---

Appendix G — Compatibility Matrix

Capability	DDI	String-based path API
Compile-time path validation	Yes — compile error on unknown segment	No — errors at runtime
Runtime dynamic path construction	No — path must be a compile-time literal	Yes
Memory allocation per access	Zero	Variable (string formatting and splitting)
Thread safety	Built-in — no allocator or lock needed	Requires allocator parameter and/or thread-specific variants
Scalar read/write	Yes	Yes
Slice length query	Yes	Yes
Slice structural mutations (insert, delete, swap, move)	No	Yes

Capability	DDI	String-based path API
Record management (add, delete, clone)	No	Yes
Editor/UI dynamic paths	No	Yes
Command system parsed paths	No	Yes
Dirty marking on write	Yes (automatic)	Yes (automatic)
Type checking	Runtime (null on mismatch)	Runtime (null on mismatch)
Nested struct traversal	Yes (unlimited depth)	Yes (unlimited depth)
Multi-database routing	Yes	Yes

[**HOWL-COMP-17-2026**] Direct Data Indexing. Github: https://github.com/ghowland/papers/tree/main/papers/COMP/H
COMP-17-2026